regression_code

Elle Strauss, Ananya Sista

2026-04-23

Package Installations

Cleaning Data

```r
# read in csv (make sure working directory is the repo)
oscar_noms <- read_csv('all_tmbd_movie_info.csv')
```

```
## Rows: 481 Columns: 17
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (6): Award, Name, Film, Genre, MPPA Rating, Director(s)
## dbl (9): Year, Ceremony, Winner, Revenue, Budget, Runtime, Vote_average_list...
## num (2): Vote_count, IMDb Rating
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
colnames(oscar_noms) <- c('year', 'ceremony', 'award', 'winner', 'film_name', 'film_producers', 'revenue

# cleaning data (removing any movies that didn't have a budget or revenue)
# ended up removing 20 observations
oscars_bp <- oscar_noms %>% filter(revenue > 0 & budget > 0)
# do log of revenue and budget (highly skewed to the right), not linear
oscars_bp$log_revenue <- log(oscars_bp$revenue)
oscars_bp$log_budget <- log(oscars_bp$budget)
oscars_bp$ROI <- oscars_bp$revenue / oscars_bp$budget

summary(oscars_bp)
```

```
##       year         ceremony        award               winner
##  Min.   :1944   Min.   :17.00   Length:461         Min.   :0.0000
##  1st Qu.:1970   1st Qu.:43.00   Class :character   1st Qu.:0.0000
##  Median :1994   Median :67.00   Mode  :character   Median :0.0000
##  Mean   :1991   Mean   :63.63                      Mean   :0.1779
##  3rd Qu.:2013   3rd Qu.:86.00                      3rd Qu.:0.0000
##  Max.   :2025   Max.   :98.00                      Max.   :1.0000
##   film_name         film_producers        revenue              budget
##  Length:461         Length:461         Min.   :6.346e+04   Min.   :   240000
```

```
##  Class :character   Class :character   1st Qu.:2.000e+07   1st Qu.:  3500000
##  Mode  :character   Mode  :character   Median :5.459e+07   Median : 14000000
##                                        Mean   :1.528e+08   Mean   : 27589998
##                                        3rd Qu.:1.650e+08   3rd Qu.: 30000000
##                                        Max.   :2.788e+09   Max.   :350000000
##     runtime      vote_average_list   vote_count        popularity
##  Min.   : 84.0   Min.   :5.500     Min.   :      6   Min.   : 0.600
##  1st Qu.:114.0   1st Qu.:7.100     1st Qu.:    225   1st Qu.: 6.808
##  Median :127.0   Median :7.400     Median :   1158   Median : 9.586
##  Mean   :131.7   Mean   :7.427     Mean   :  66557   Mean   :14.329
##  3rd Qu.:145.0   3rd Qu.:7.800     3rd Qu.:   7311   3rd Qu.:19.000
##  Max.   :248.0   Max.   :8.600     Max.   :1200000   Max.   :80.000
##     genre            imdb_rating   rotten_tomato_score mppa_rating
##  Length:461         Min.   :6.10   Min.   :  9.70      Length:461
##  Class :character   1st Qu.:7.30   1st Qu.: 85.00      Class :character
##  Mode  :character   Median :7.70   Median : 91.00      Mode  :character
##                     Mean   :7.66   Mean   : 88.14
##                     3rd Qu.:8.00   3rd Qu.: 95.00
##                     Max.   :9.30   Max.   :100.00
##   directors          log_revenue      log_budget         ROI
##  Length:461         Min.   :11.06   Min.   :12.39   Min.   :  0.04533
##  Class :character   1st Qu.:16.81   1st Qu.:15.07   1st Qu.:  2.69068
##  Mode  :character   Median :17.82   Median :16.45   Median :  4.97088
##                     Mean   :17.79   Mean   :16.22   Mean   :  8.82163
##                     3rd Qu.:18.92   3rd Qu.:17.22   3rd Qu.:  9.50000
##                     Max.   :21.75   Max.   :19.67   Max.   :180.18018
```

```r
# confirm winner is a numeric
oscars_bp$winner <- factor(oscars_bp$winner)
```

Separating Out Critic, Audience, and Market Variables

```r
critics_vars <- c("rotten_tomato_score")

audience_vars <- c("imdb_rating", "vote_average_list", "vote_count")

market_vars <- c("log_revenue", "log_budget", "popularity", "ROI")
```

Logistic Regression

Individual Predictors

```r
AIC1.1 <- matrix(rep(0,9), ncol=1)
rownames(AIC1.1) <- c("log_revenue", "log_budget", "runtime", "vote_average_list", "vote_count", "popula

mod1.1a <- glm(winner ~ log_revenue, family=binomial, data=oscars_bp)
summary(mod1.1a)
```

```
##
## Call:
## glm(formula = winner ~ log_revenue, family = binomial, data = oscars_bp)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.31619    1.47927  -3.594 0.000326 ***
## log_revenue  0.21082    0.08137   2.591 0.009578 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 424.63  on 459  degrees of freedom
## AIC: 428.63
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[1] <- AIC(mod1.1a)
predict1.1a <- predict(mod1.1a, type="response")
oscars1.1a <- ifelse(predict1.1a >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1a)
```

```
##    oscars1.1a
##       0
##   0 379
##   1  82
```

```r
mod1.1b <- glm(winner ~ log_budget, family=binomial, data=oscars_bp)
summary(mod1.1b)
```

```
##
## Call:
## glm(formula = winner ~ log_budget, family = binomial, data = oscars_bp)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.19356    1.35794  -0.143    0.887
## log_budget  -0.08273    0.08396  -0.985    0.324
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 430.67  on 459  degrees of freedom
## AIC: 434.67
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[2] <- AIC(mod1.1b)
predict1.1b <- predict(mod1.1b, type="response")
oscars1.1b <- ifelse(predict1.1b >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1b)
```

```
##    oscars1.1b
##       0
##   0 379
##   1  82
```

```r
mod1.1c <- glm(winner ~ runtime, family=binomial, data=oscars_bp)
summary(mod1.1c)
```

```
##
## Call:
## glm(formula = winner ~ runtime, family = binomial, data = oscars_bp)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.075708   0.608950  -5.051  4.4e-07 ***
## runtime      0.011511   0.004364   2.638  0.00835 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 424.88  on 459  degrees of freedom
## AIC: 428.88
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[3] <- AIC(mod1.1c)
predict1.1c <- predict(mod1.1c, type="response")
oscars1.1c <- ifelse(predict1.1c >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1c)
```

```
##    oscars1.1c
##       0
##   0 379
##   1  82
```

```r
mod1.1d <- glm(winner ~ vote_average_list, family=binomial, data=oscars_bp)
summary(mod1.1d)
```

```
##
## Call:
## glm(formula = winner ~ vote_average_list, family = binomial,
##     data = oscars_bp)
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -9.8997     2.0130  -4.918 8.75e-07 ***
## vote_average_list   1.1136     0.2645   4.210 2.55e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 412.37  on 459  degrees of freedom
## AIC: 416.37
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[4] <- AIC(mod1.1d)
predict1.1d <- predict(mod1.1d, type="response")
oscars1.1d <- ifelse(predict1.1d >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1d)
```

```
##    oscars1.1d
##       0
##   0 379
##   1  82
```

```r
mod1.1e <- glm(winner ~ vote_count, family=binomial, data=oscars_bp)
summary(mod1.1e)
```

```
##
## Call:
## glm(formula = winner ~ vote_count, family = binomial, data = oscars_bp)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.505e+00  1.296e-01 -11.616   <2e-16 ***
## vote_count  -4.083e-07  7.528e-07  -0.542    0.588
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 431.33  on 459  degrees of freedom
## AIC: 435.33
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[5] <- AIC(mod1.1e)
predict1.1e <- predict(mod1.1e, type="response")
oscars1.1e <- ifelse(predict1.1e >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1e)
```

```
##    oscars1.1e
##       0
##   0 379
##   1  82
```

```r
mod1.1f <- glm(winner ~ popularity, family=binomial, data=oscars_bp)
summary(mod1.1f)
```

```
##
## Call:
## glm(formula = winner ~ popularity, family = binomial, data = oscars_bp)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.614743   0.188875  -8.549   <2e-16 ***
## popularity   0.005749   0.009708   0.592    0.554
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 431.30  on 459  degrees of freedom
## AIC: 435.3
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[6] <- AIC(mod1.1f)
predict1.1f <- predict(mod1.1f, type="response")
oscars1.1f <- ifelse(predict1.1f >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1f)
```

```
##    oscars1.1f
##       0
##   0 379
##   1  82
```

```r
mod1.1g <- glm(winner ~ imdb_rating, family=binomial, data=oscars_bp)
summary(mod1.1g)
```

```
##
## Call:
## glm(formula = winner ~ imdb_rating, family = binomial, data = oscars_bp)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -10.7373     2.0553  -5.224 1.75e-07 ***
## imdb_rating   1.1885     0.2622   4.533 5.83e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 409.80  on 459  degrees of freedom
## AIC: 413.8
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[7] <- AIC(mod1.1g)
predict1.1g <- predict(mod1.1g, type="response")
oscars1.1g <- ifelse(predict1.1g >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1g)
```

```
##    oscars1.1g
##       0   1
##   0 378   1
##   1  81   1
```

```r
mod1.1h <- glm(winner ~ rotten_tomato_score, family=binomial, data=oscars_bp)
summary(mod1.1h)
```

```
##
## Call:
## glm(formula = winner ~ rotten_tomato_score, family = binomial,
##     data = oscars_bp)
##
## Coefficients:
##                      Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -1.868810   1.023338  -1.826   0.0678 .
## rotten_tomato_score  0.003828   0.011489   0.333   0.7390
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 431.52  on 459  degrees of freedom
## AIC: 435.52
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[8] <- AIC(mod1.1h)
predict1.1h <- predict(mod1.1h, type="response")
oscars1.1h <- ifelse(predict1.1h >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1h)
```

```
##    oscars1.1h
##       0
##   0 379
##   1  82
```

```r
mod1.1i <- glm(winner ~ ROI, family=binomial, data=oscars_bp)
summary(mod1.1i)
```

```
##
## Call:
## glm(formula = winner ~ ROI, family = binomial, data = oscars_bp)
##
## Coefficients:
```

```
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.716070   0.146624 -11.704   <2e-16 ***
## ROI          0.019012   0.007852   2.421   0.0155 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 431.64  on 460  degrees of freedom
## Residual deviance: 425.26  on 459  degrees of freedom
## AIC: 429.26
##
## Number of Fisher Scoring iterations: 4
```

```r
AIC1.1[9] <- AIC(mod1.1i)
predict1.1i <- predict(mod1.1i, type="response")
oscars1.1i <- ifelse(predict1.1i >= 0.5,1,0)
table(oscars_bp$winner, oscars1.1i)
```

```
##    oscars1.1i
##       0   1
##   0 378   1
##   1  81   1
```

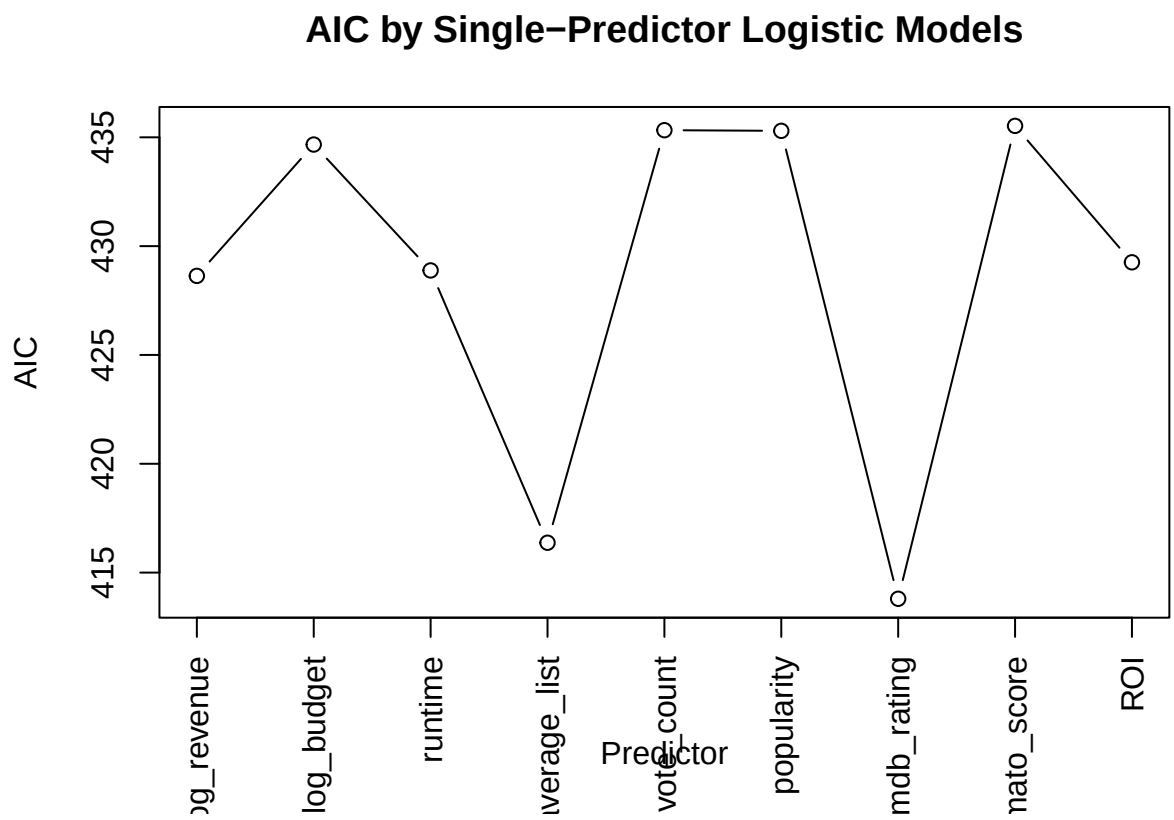
```r
AIC1.1
```

```
##                         [,1]
## log_revenue         428.6331
## log_budget          434.6680
## runtime             428.8828
## vote_average_list   416.3723
## vote_count          435.3265
## popularity          435.2977
## imdb_rating         413.7970
## rotten_tomato_score 435.5250
## ROI                 429.2567
```

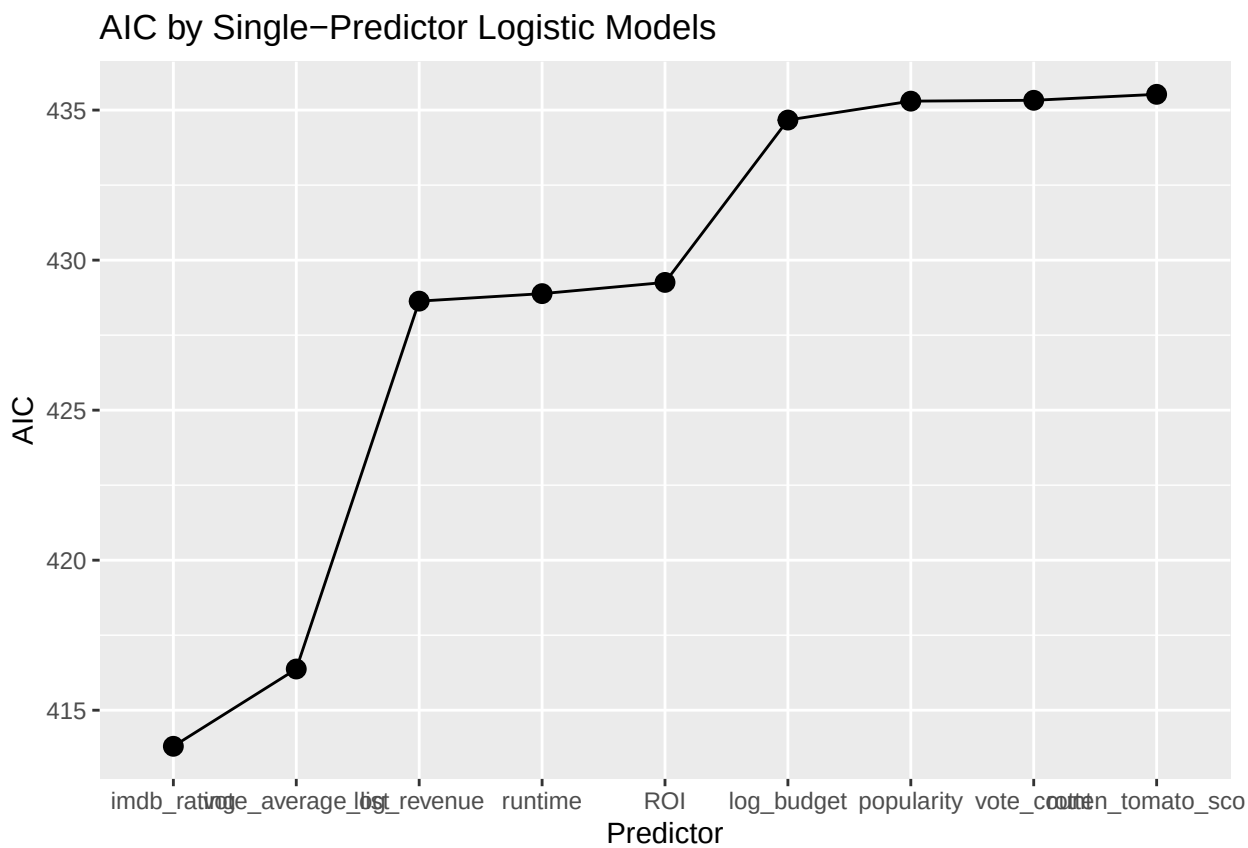
Plots

```r
aic_df <- data.frame(
  predictor = rownames(AIC1.1),
  AIC = AIC1.1[,1]
)

plot(
  aic_df$AIC,
  type = "b",
  xaxt = "n",
  xlab = "Predictor",
  ylab = "AIC",
  main = "AIC by Single-Predictor Logistic Models"
```

```
)

axis(1, at = 1:nrow(aic_df), labels = aic_df$predictor, las = 2)
```




```
ggplot(aic_df, aes(x = reorder(predictor, AIC), y = AIC)) +
  geom_point(size = 3) +
  geom_line(group = 1) +
  labs(
    title = "AIC by Single-Predictor Logistic Models",
    x = "Predictor",
    y = "AIC"
  )
```



Comparing Models (All Predictors)

```
mod.full <- glm(winner ~ log_revenue + log_budget + runtime + vote_average_list + vote_count + populari
mod.null <- glm(winner ~ 1, family=binomial, data=oscars_bp) # intercept only model

stepAIC(mod.full, direction="backward")
```

```
## Start:  AIC=395.95
## winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     vote_count + popularity + imdb_rating + rotten_tomato_score +
##     ROI
##
##                       Df Deviance    AIC
## - vote_count           1   376.01 394.01
## - imdb_rating          1   376.22 394.22
## - rotten_tomato_score  1   376.61 394.61
## - popularity           1   376.95 394.95
## - vote_average_list    1   377.21 395.21
## - ROI                  1   377.61 395.61
## <none>                     375.95 395.95
## - runtime              1   383.89 401.89
## - log_budget           1   392.50 410.50
## - log_revenue          1   394.10 412.10
```

```
##
## Step:  AIC=394.01
## winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     popularity + imdb_rating + rotten_tomato_score + ROI
##
##                       Df Deviance    AIC
## - imdb_rating          1   376.23 392.23
## - rotten_tomato_score  1   376.67 392.67
## - vote_average_list    1   377.39 393.39
## - ROI                  1   377.77 393.77
## - popularity           1   377.81 393.81
## <none>                     376.01 394.01
## - runtime              1   383.89 399.89
## - log_budget           1   392.94 408.94
## - log_revenue          1   394.12 410.12
##
## Step:  AIC=392.23
## winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     popularity + rotten_tomato_score + ROI
##
##                       Df Deviance    AIC
## - rotten_tomato_score  1   376.77 390.77
## - ROI                  1   377.88 391.88
## <none>                     376.23 392.23
## - popularity           1   378.55 392.55
## - runtime              1   384.37 398.37
## - vote_average_list    1   387.89 401.89
## - log_budget           1   393.02 407.02
## - log_revenue          1   395.31 409.31
##
## Step:  AIC=390.77
## winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     popularity + ROI
##
##                     Df Deviance    AIC
## - ROI                1   378.55 390.55
## <none>                   376.77 390.77
## - popularity         1   379.14 391.14
## - runtime            1   385.59 397.59
## - vote_average_list  1   388.00 400.00
## - log_budget         1   393.37 405.37
## - log_revenue        1   396.54 408.54
##
## Step:  AIC=390.55
## winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     popularity
##
##                     Df Deviance    AIC
## <none>                   378.55 390.55
## - popularity         1   381.41 391.41
## - runtime            1   387.19 397.19
## - vote_average_list  1   391.32 401.32
## - log_budget         1   398.12 408.12
## - log_revenue        1   402.72 412.72
```

```
##
## Call:  glm(formula = winner ~ log_revenue + log_budget + runtime + vote_average_list +
##     popularity, family = binomial, data = oscars_bp)
##
## Coefficients:
##       (Intercept)        log_revenue         log_budget            runtime
##         -12.74459            0.66372           -0.66740            0.01505
## vote_average_list         popularity
##           1.11952           -0.02439
##
## Degrees of Freedom: 460 Total (i.e. Null);  455 Residual
## Null Deviance:       431.6
## Residual Deviance: 378.5     AIC: 390.5
```

```r
# best backward model: log_revenue + log_budget + runtime + vote_average_list + popularity
stepAIC(mod.null, direction="forward", scope=list(upper=mod.full, lower=mod.null))
```

```
## Start:  AIC=433.64
## winner ~ 1
##
##                        Df Deviance    AIC
## + imdb_rating           1   409.80 413.80
## + vote_average_list     1   412.37 416.37
## + log_revenue           1   424.63 428.63
## + runtime               1   424.88 428.88
## + ROI                   1   425.26 429.26
## <none>                      431.64 433.64
## + log_budget            1   430.67 434.67
## + popularity            1   431.30 435.30
## + vote_count            1   431.33 435.33
## + rotten_tomato_score   1   431.52 435.52
##
## Step:  AIC=413.8
## winner ~ imdb_rating
##
##                        Df Deviance    AIC
## + ROI                   1   405.41 411.41
## + runtime               1   406.67 412.67
## + log_budget            1   407.45 413.45
## + log_revenue           1   407.69 413.69
## <none>                      409.80 413.80
## + rotten_tomato_score   1   408.30 414.30
## + popularity            1   408.38 414.38
## + vote_count            1   408.95 414.95
## + vote_average_list     1   409.61 415.61
##
## Step:  AIC=411.41
## winner ~ imdb_rating + ROI
##
##                        Df Deviance    AIC
## + runtime               1   400.78 408.78
## <none>                      405.41 411.41
## + rotten_tomato_score   1   403.66 411.66
## + popularity            1   403.74 411.74
```

```
## + log_budget           1   404.31 412.31
## + log_revenue          1   404.42 412.42
## + vote_count           1   404.66 412.66
## + vote_average_list    1   404.97 412.97
##
## Step:  AIC=408.78
## winner ~ imdb_rating + ROI + runtime
##
##                       Df Deviance    AIC
## + log_budget           1   397.13 407.13
## + popularity           1   398.48 408.48
## <none>                     400.78 408.78
## + vote_count           1   399.94 409.94
## + rotten_tomato_score  1   400.03 410.03
## + vote_average_list    1   400.30 410.30
## + log_revenue          1   400.52 410.52
##
## Step:  AIC=407.13
## winner ~ imdb_rating + ROI + runtime + log_budget
##
##                       Df Deviance    AIC
## + log_revenue          1   379.06 391.06
## <none>                     397.13 407.13
## + rotten_tomato_score  1   395.44 407.44
## + vote_average_list    1   396.68 408.68
## + popularity           1   396.76 408.76
## + vote_count           1   397.03 409.03
##
## Step:  AIC=391.06
## winner ~ imdb_rating + ROI + runtime + log_budget + log_revenue
##
##                       Df Deviance    AIC
## <none>                     379.06 391.06
## + popularity           1   378.12 392.12
## + rotten_tomato_score  1   378.25 392.25
## + vote_average_list    1   378.62 392.62
## + vote_count           1   378.74 392.74
```

```
##
## Call:  glm(formula = winner ~ imdb_rating + ROI + runtime + log_budget +
##     log_revenue, family = binomial, data = oscars_bp)
##
## Coefficients:
## (Intercept)  imdb_rating          ROI      runtime   log_budget  log_revenue
##    -9.30388      0.84017     -0.02097      0.01593     -0.98822      0.85203
##
## Degrees of Freedom: 460 Total (i.e. Null);  455 Residual
## Null Deviance:       431.6
## Residual Deviance: 379.1     AIC: 391.1
```

```
# best forward model: imdb_rating + ROI + runtime + log_budget + log_revenue
```

Predicting Based on Best Fit

Adding Test and Train Data

```
set.seed(123)

train_idx <- sample(1:nrow(oscars_bp), 0.7 * nrow(oscars_bp))

train <- oscars_bp[train_idx, ]
test  <- oscars_bp[-train_idx, ]
```

```
# Try out the best fit on predicting (use backwards)
bestFitBack <- glm(winner ~ log_revenue + log_budget + runtime + vote_average_list + popularity, family=
pred_prob_back <- predict(bestFitBack, newdata = test, type = "response")
pred_back <- ifelse(pred_prob_back >= 0.5, 1, 0)

table(Actual = test$winner, Predicted = pred_back)
```

```
##       Predicted
## Actual   0   1
##      0 110   3
##      1  24   2
```

```
# oscars_bp$predict_BestBack <- predict(bestFitBack, type="response")
# oscars.1BestBack <- oscars_bp %>% group_by(year) %>% mutate(predicted = 0, predicted = replace(predic
# table(oscars.1BestBack$winner, oscars.1BestBack$predicted)

bestFitFor <- glm(winner ~ imdb_rating + ROI + runtime + log_budget + log_revenue, family=binomial, data
pred_prob_for <- predict(bestFitFor, newdata = test, type = "response")
pred_for <- ifelse(pred_prob_for >= 0.5, 1, 0)

table(Actual = test$winner, Predicted = pred_for)
```

```
##       Predicted
## Actual   0   1
##      0 109   4
##      1  24   2
```

```
# oscars_bp$predict_BestFor <- predict(bestFitFor, type="response")
# oscars.1BestFor <- oscars_bp %>% group_by(year) %>% mutate(predicted = 0, predicted = replace(predict
# table(oscars.1BestBack$winner, oscars.1BestFor$predicted)
```

Model Comparisons

```
# Critics only model
model_critics <- glm(
  winner ~ rotten_tomato_score,
  family = binomial,
  data = train
```

```r
)

# Audience only model
model_audience <- glm(
  winner ~ imdb_rating + vote_average_list + vote_count,
  family = binomial,
  data = train
)

# Combined model (critics + audience)
model_crit_aud <- glm(
  winner ~ rotten_tomato_score +
    imdb_rating + vote_average_list + vote_count,
  family = binomial,
  data = train
)

# Market model (bugdet, popularity, revenue)
model_market <- glm(winner ~ log_revenue + log_budget + popularity,
                    family = binomial, data = train)

# Full model (controls included)
model_full <- glm(
  winner ~ rotten_tomato_score +
    imdb_rating + vote_average_list + vote_count +
    log_revenue + log_budget + runtime + popularity,
  family = binomial,
  data = train
)



AIC(model_critics, model_audience, model_crit_aud, model_market, model_full)
```

```
##                df      AIC
## model_critics   2 301.5092
## model_audience  4 288.5708
## model_crit_aud  5 289.0810
## model_market    4 278.8338
## model_full      9 266.3122
```

```r
# predicted probabilities
p_critics <- predict(model_critics, newdata = test, type = "response")
p_audience <- predict(model_audience, newdata = test, type = "response")
p_crit_aud <- predict(model_crit_aud, newdata = test, type = "response")
p_market <- predict(model_market, newdata = test, type = "response")
p_full <- predict(model_full, newdata = test, type = "response")

# ROC curves
roc_critics <- roc(test$winner, p_critics)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
roc_audience <- roc(test$winner, p_audience)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
roc_crit_aud <- roc(test$winner, p_crit_aud)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

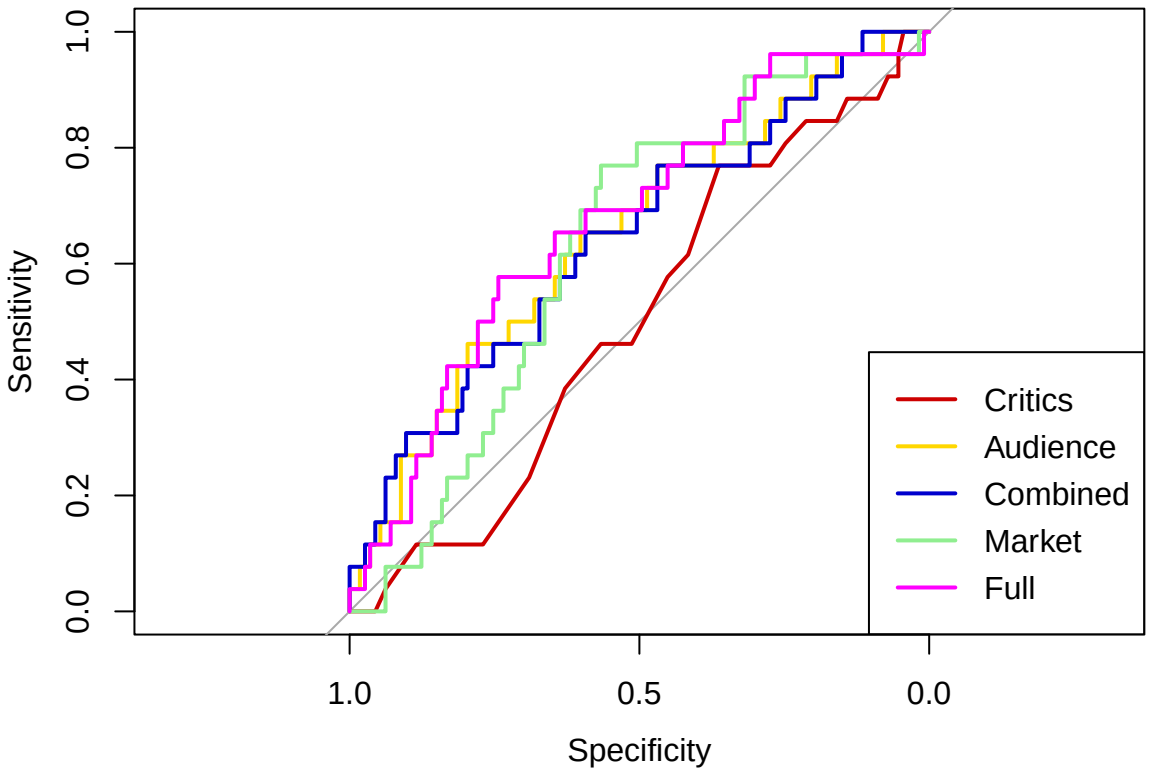
```r
roc_market <- roc(test$winner, p_market)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
roc_full <- roc(test$winner, p_full)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
plot(roc_critics, col="red3")
lines(roc_audience, col="gold")
lines(roc_crit_aud, col="blue3")
lines(roc_market, col="lightgreen")
lines(roc_full, col="magenta")


legend("bottomright",
       legend=c("Critics", "Audience", "Combined", "Market", "Full"),
       col=c("red3","gold","blue3", "lightgreen", "magenta"),
       lwd=2)
```




```r
# AUC values
auc(roc_critics)
```

```
## Area under the curve: 0.502
```

```r
auc(roc_audience)
```

```
## Area under the curve: 0.6494
```

```r
auc(roc_crit_aud)
```

```
## Area under the curve: 0.643
```

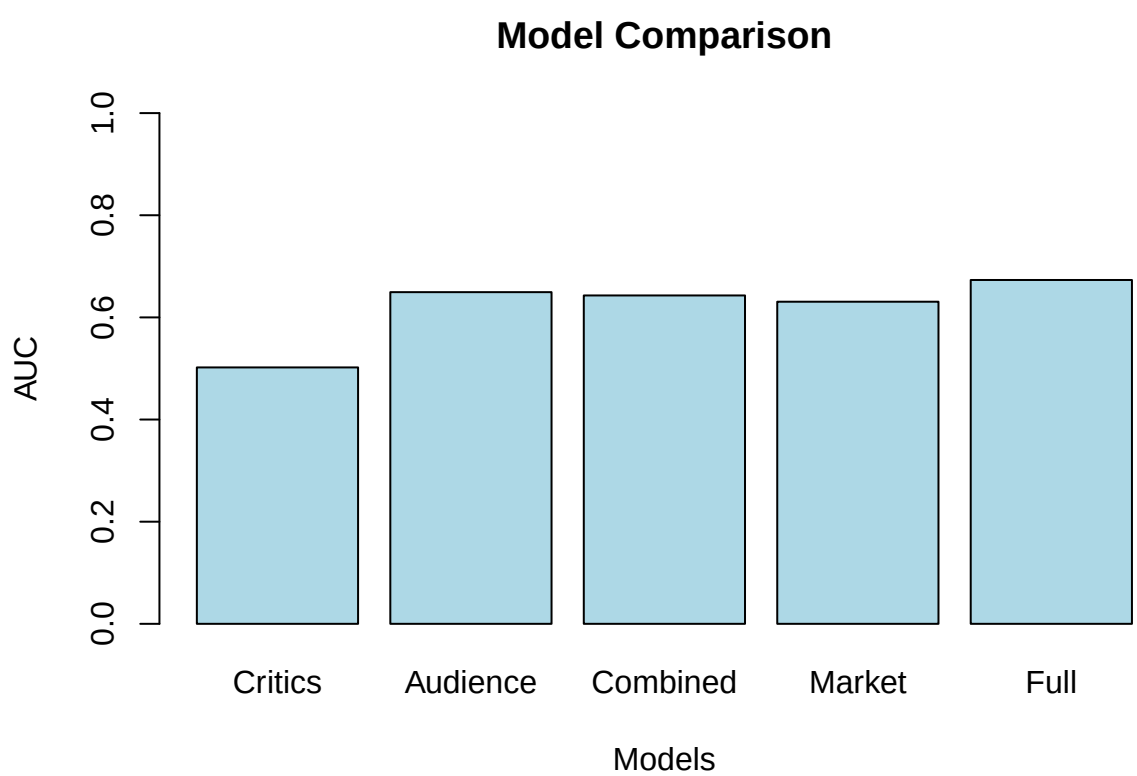
```r
auc(roc_market)
```

```
## Area under the curve: 0.6307
```

```r
auc(roc_full)
```

```
## Area under the curve: 0.6732
```

```r
auc_df <- data.frame(
  model = c("Critics","Audience","Combined","Market","Full"),
  auc = c(auc(roc_critics),
          auc(roc_audience),
          auc(roc_crit_aud),
          auc(roc_market),
          auc(roc_full))
)

barplot(auc_df$auc, names.arg = auc_df$model, main="Model Comparison", ylim=c(0, 1), col="lightblue", xl
```



Statistical Test of Models

```r
anova(model_audience, model_crit_aud, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: winner ~ imdb_rating + vote_average_list + vote_count
## Model 2: winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       318     280.57
## 2       317     279.08  1   1.4898   0.2222
```

```r
anova(model_critics, model_crit_aud, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: winner ~ rotten_tomato_score
## Model 2: winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count
##   Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
## 1       320     297.51
## 2       317     279.08  3   18.428 0.0003589 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
anova(model_market, model_full, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: winner ~ log_revenue + log_budget + popularity
## Model 2: winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count + log_revenue + log_budget + runtime + popularity
##   Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
## 1       318     270.83
## 2       313     248.31  5   22.522 0.0004166 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
anova(model_crit_aud, model_full, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count
## Model 2: winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count + log_revenue + log_budget + runtime + popularity
##   Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
## 1       317     279.08
## 2       313     248.31  4   30.769 3.412e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Predictions
pred_critics <- ifelse(p_critics >= 0.5, 1, 0)
pred_audience <- ifelse(p_audience >= 0.5, 1, 0)
pred_crit_aud <- ifelse(p_crit_aud >= 0.5, 1, 0)
pred_market <- ifelse(p_market >= 0.5, 1, 0)

# Confusion matrices
table(test$winner, pred_critics)
```

```
##    pred_critics
##       0
##   0 113
##   1  26
```

```r
table(test$winner, pred_audience)
```

```
##    pred_audience
##       0   1
##   0 113   0
##   1  25   1
```

```r
table(test$winner, pred_crit_aud)
```

```
##    pred_crit_aud
##       0   1
##   0 113   0
##   1  24   2
```

```r
table(test$winner, pred_market)
```

```
##    pred_market
##       0   1
##   0 109   4
##   1  26   0
```

```r
summary(model_crit_aud)
```

```
##
## Call:
## glm(formula = winner ~ rotten_tomato_score + imdb_rating + vote_average_list +
##     vote_count, family = binomial, data = train)
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -1.101e+01  2.544e+00  -4.326 1.52e-05 ***
## rotten_tomato_score -1.679e-02  1.340e-02  -1.253    0.210
## imdb_rating          9.221e-01  7.702e-01   1.197    0.231
## vote_average_list    5.165e-01  7.788e-01   0.663    0.507
## vote_count          -6.702e-07  9.217e-07  -0.727    0.467
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 297.55  on 321  degrees of freedom
## Residual deviance: 279.08  on 317  degrees of freedom
## AIC: 289.08
##
## Number of Fisher Scoring iterations: 4
```

```r
exp(cbind(Odds_Ratio = coef(model_crit_aud),
          confint(model_crit_aud)))
```

```
## Waiting for profiling to be done...
```

```
##                       Odds_Ratio        2.5 %       97.5 %
## (Intercept)         1.661741e-05 9.274901e-08  0.002072253
## rotten_tomato_score 9.833524e-01 9.579092e-01  1.010925931
## imdb_rating         2.514687e+00 5.573814e-01 11.535302270
## vote_average_list   1.676076e+00 3.718027e-01  7.916667792
## vote_count          9.999993e-01 9.999973e-01  1.000001030
```

Confusion Table

Left to Right
True Negative: Movie did not win, model correctly predicted no win.
False Positive: Movie lost, but model predicted winner.
False Negative: Movie actually won, but model predicted loser.
True Positive: Movie won, model correctly predicted winner.

Comparing Step Models

```r
roc_back <- roc(test$winner, pred_prob_back)
```

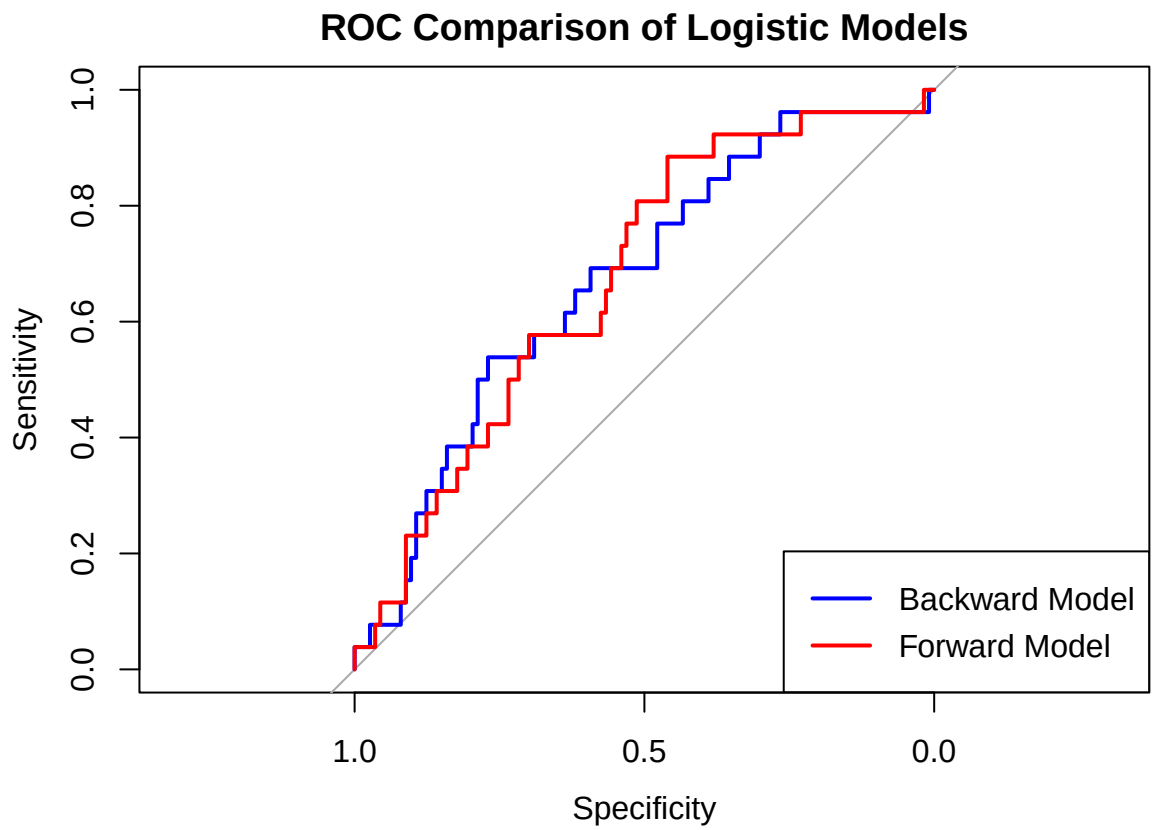
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

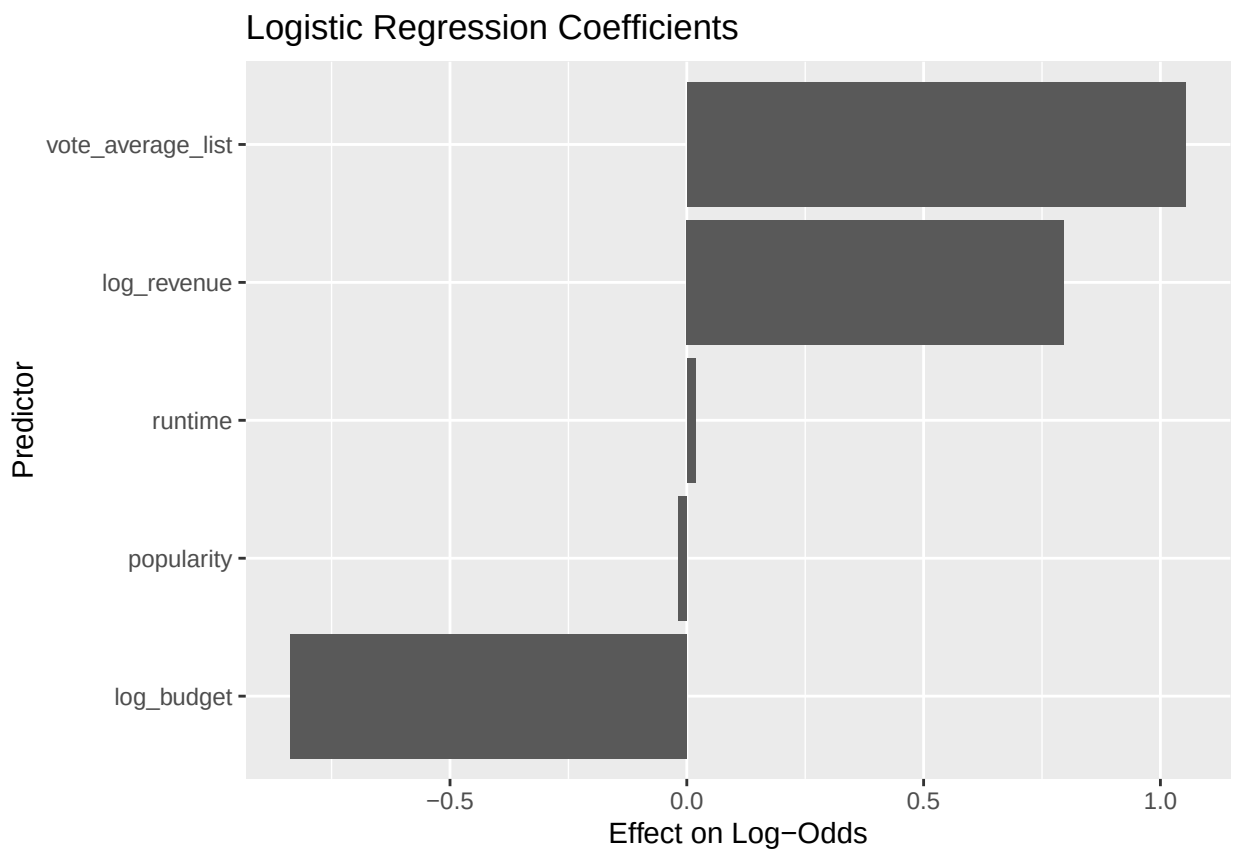
```r
roc_for  <- roc(test$winner, pred_prob_for)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
plot(roc_back, col="blue", main="ROC Comparison of Logistic Models")
lines(roc_for, col="red")

legend("bottomright",
       legend=c("Backward Model", "Forward Model"),
       col=c("blue", "red"),
       lwd=2)
```

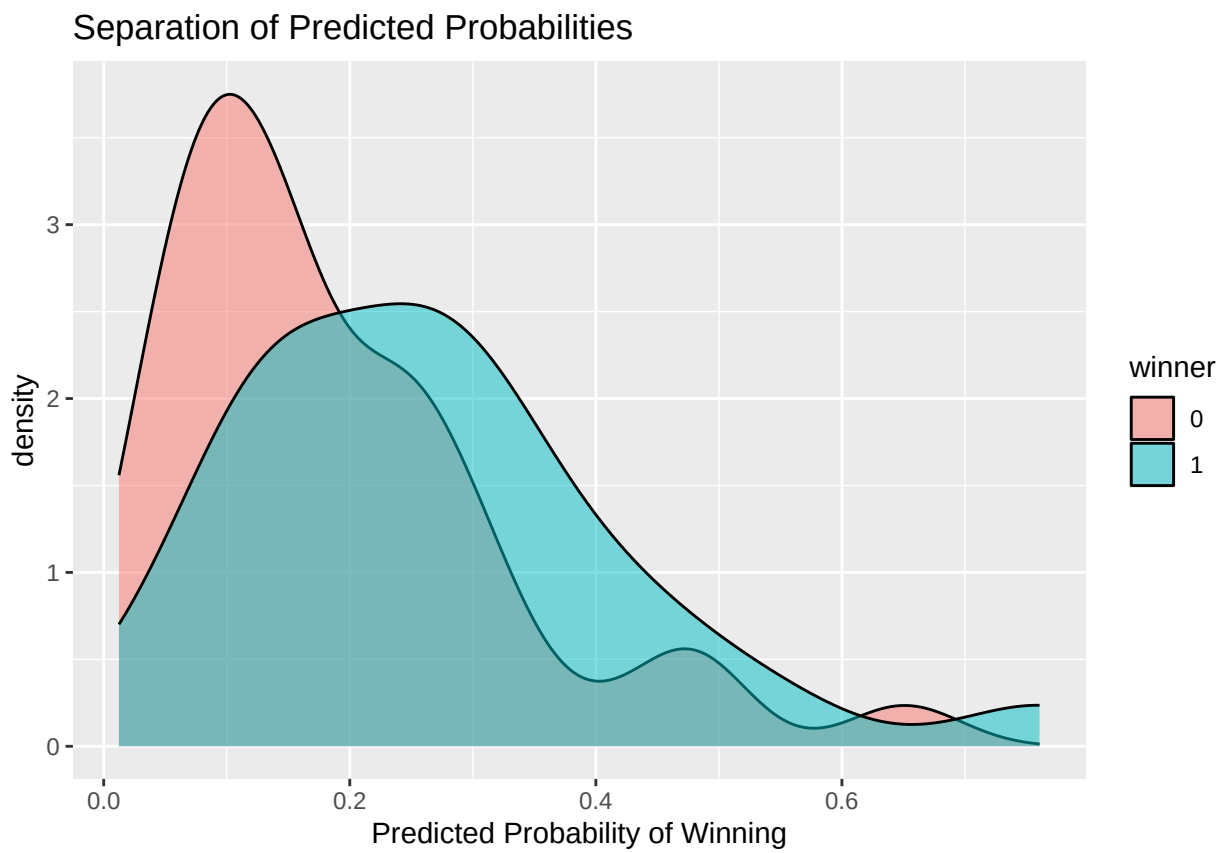



```r
# Coefficients Odds Ratio
# visualizes the estimated effects of predictor variables on a categorical outcome by displaying their
coef_df <- data.frame(
  variable = names(coef(bestFitBack)),
  value = coef(bestFitBack)
)

ggplot(coef_df[-1,], aes(x = reorder(variable, value), y = value)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(
    title = "Logistic Regression Coefficients",
    x = "Predictor",
    y = "Effect on Log-Odds"
  )
```



Probability Of Separation

```
ggplot(test, aes(x = pred_prob_back, fill = winner)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Separation of Predicted Probabilities",
    x = "Predicted Probability of Winning"
  )
```



Does the model actually separate winners from losers? If curves overlap → weak model
If separated → strong model

Classification Tree

```
# Classification Tree
tree_model <- rpart(winner ~ log_revenue + log_budget + imdb_rating + popularity + vote_count + runtime
tree_pred <- predict(tree_model, newdata = test, type="class")
table(Actual=test$winner, Predicted=tree_pred)
```

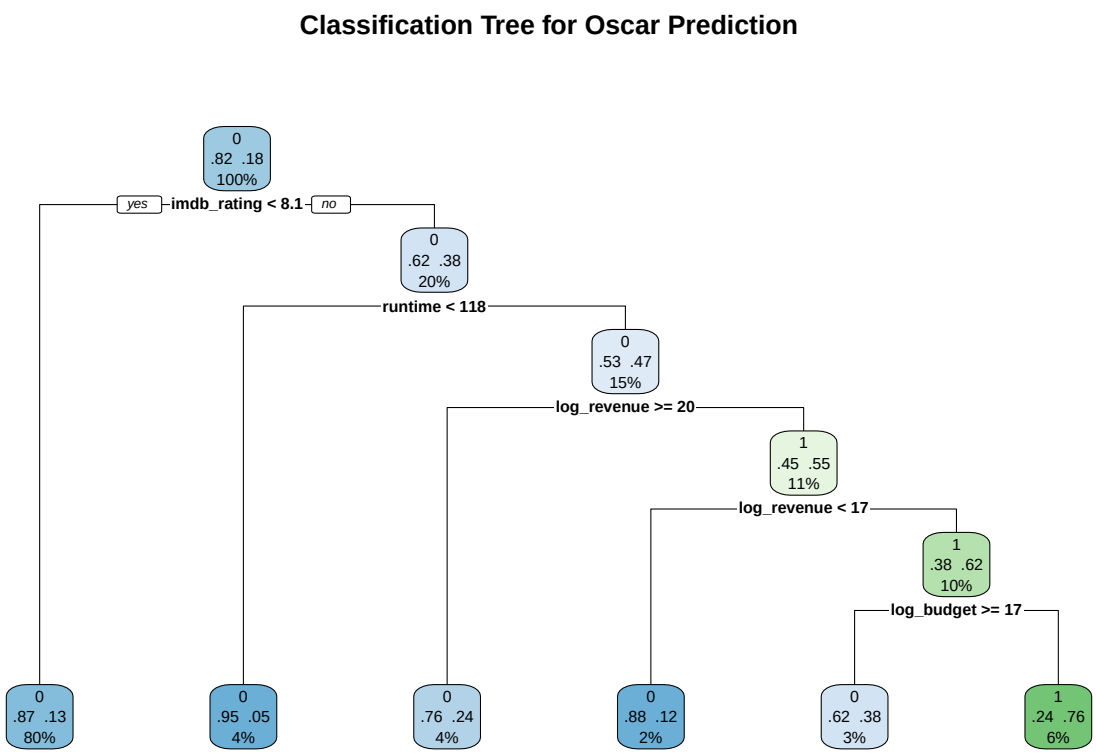
```
##       Predicted
## Actual   0   1
##      0 108   5
##      1  20   6
```

```
print(tree_model)
```

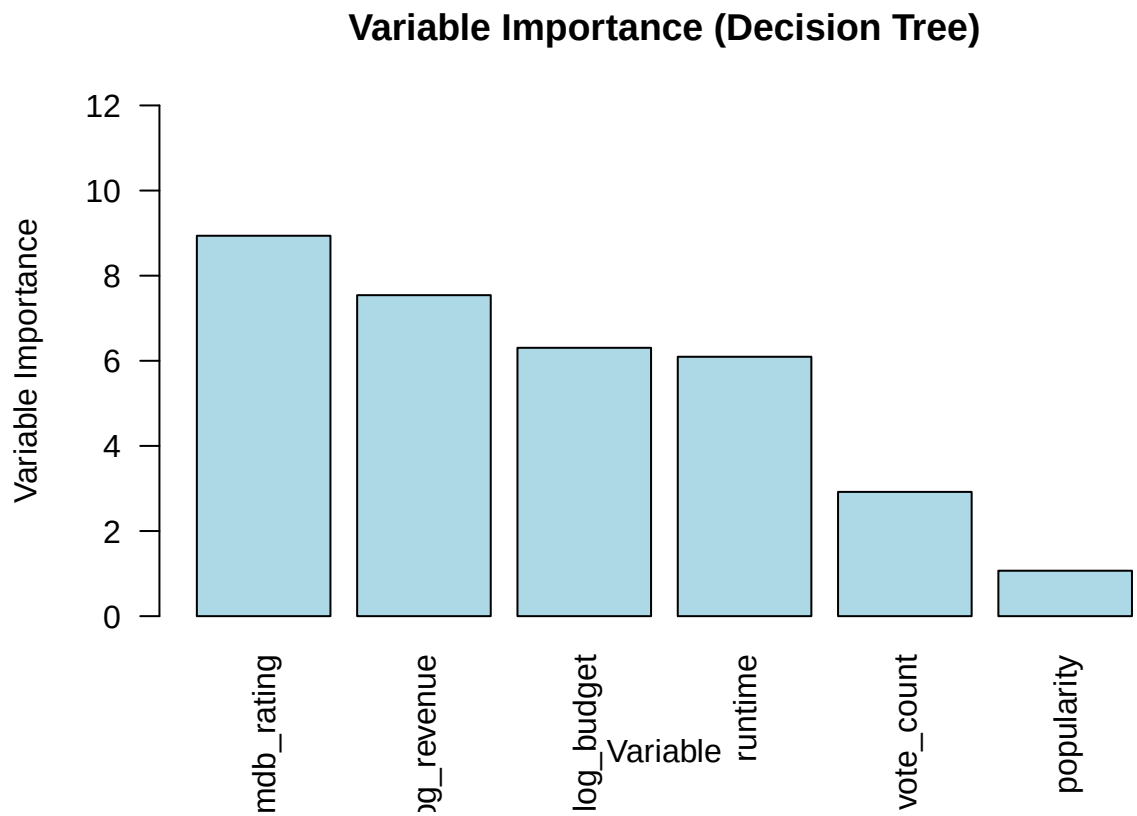
```
## n= 461
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##  1) root 461 82 0 (0.8221258 0.1778742)
```

```
##    2) imdb_rating< 8.05 371 48 0 (0.8706199 0.1293801) *
##    3) imdb_rating>=8.05 90 34 0 (0.6222222 0.3777778)
##      6) runtime< 118 20  1 0 (0.9500000 0.0500000) *
##      7) runtime>=118 70 33 0 (0.5285714 0.4714286)
##       14) log_revenue>=19.72197 17  4 0 (0.7647059 0.2352941) *
##       15) log_revenue< 19.72197 53 24 1 (0.4528302 0.5471698)
##         30) log_revenue< 16.96494 8  1 0 (0.8750000 0.1250000) *
##         31) log_revenue>=16.96494 45 17 1 (0.3777778 0.6222222)
##           62) log_budget>=16.9927 16  6 0 (0.6250000 0.3750000) *
##           63) log_budget< 16.9927 29  7 1 (0.2413793 0.7586207) *
```

```r
# Classification Tree Plot
rpart.plot(tree_model,
           type = 2,
           extra = 104,
           main = "Classification Tree for Oscar Prediction")
```




```r
# Variable Importance
barplot(tree_model$variable.importance,
        main = "Variable Importance (Decision Tree)",
        las = 2, ylim=c(0, 12), col="lightblue", ylab="Variable Importance", xlab="Variable")
```



LDA

```
model <- lda(winner ~ revenue + budget + imdb_rating + popularity, data=train)

lda_pred <- predict(model, newdata=test)$class

table(Actual=test$winner, Predicted=lda_pred)
```

```
##       Predicted
## Actual   0   1
##      0 107   6
##      1  24   2
```

General Analysis

```
# Means
mean(pred_back == test$winner)
```

```
## [1] 0.8057554
```

```r
mean(lda_pred == test$winner)
```

```
## [1] 0.7841727
```

```r
mean(tree_pred == test$winner)
```

```
## [1] 0.8201439
```

```r
# Confusion Matrix
confusionMatrix(as.factor(pred_back), test$winner)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 110  24
##          1   3   2
##
##                Accuracy : 0.8058
##                  95% CI : (0.7301, 0.8679)
##     No Information Rate : 0.8129
##     P-Value [Acc > NIR] : 0.6354645
##
##                   Kappa : 0.0731
##
##  Mcnemar's Test P-Value : 0.0001186
##
##             Sensitivity : 0.97345
##             Specificity : 0.07692
##          Pos Pred Value : 0.82090
##          Neg Pred Value : 0.40000
##              Prevalence : 0.81295
##          Detection Rate : 0.79137
##    Detection Prevalence : 0.96403
##       Balanced Accuracy : 0.52519
##
##        'Positive' Class : 0
##
```

```r
confusionMatrix(as.factor(lda_pred), test$winner)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 107  24
##          1   6   2
##
##                Accuracy : 0.7842
##                  95% CI : (0.7065, 0.8494)
##     No Information Rate : 0.8129
```

```
##     P-Value [Acc > NIR] : 0.836566
##
##                   Kappa : 0.0325
##
##  Mcnemar's Test P-Value : 0.001911
##
##             Sensitivity : 0.94690
##             Specificity : 0.07692
##          Pos Pred Value : 0.81679
##          Neg Pred Value : 0.25000
##              Prevalence : 0.81295
##          Detection Rate : 0.76978
##    Detection Prevalence : 0.94245
##       Balanced Accuracy : 0.51191
##
##        'Positive' Class : 0
##
```

```r
confusionMatrix(as.factor(tree_pred), test$winner)
```

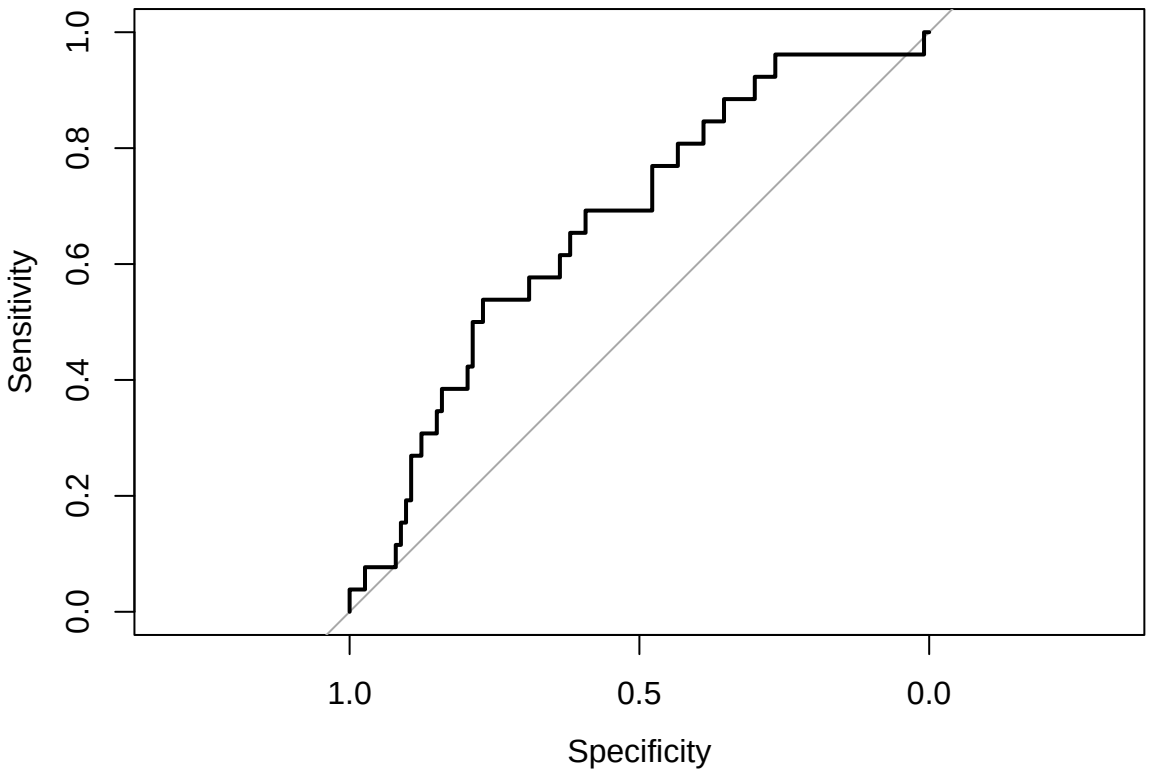
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 108  20
##          1   5   6
##
##                Accuracy : 0.8201
##                  95% CI : (0.7461, 0.8801)
##     No Information Rate : 0.8129
##     P-Value [Acc > NIR] : 0.46570
##
##                   Kappa : 0.2398
##
##  Mcnemar's Test P-Value : 0.00511
##
##             Sensitivity : 0.9558
##             Specificity : 0.2308
##          Pos Pred Value : 0.8438
##          Neg Pred Value : 0.5455
##              Prevalence : 0.8129
##          Detection Rate : 0.7770
##    Detection Prevalence : 0.9209
##       Balanced Accuracy : 0.5933
##
##        'Positive' Class : 0
##
```

```r
# ROC/AUC
# A probability curve that plots the True Positive Rate (Sensitivity)
roc_obj <- roc(test$winner, pred_prob_back)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
auc(roc_obj)
```

```
## Area under the curve: 0.6712
```

```r
plot(roc_obj)
```